

Best Fit Line

today we are going to write our own code for fitting a line and then compare it to different functions in python. We are going to take our math from Daniel C. Harris, Quantitative Chemical Analysis pages 66 and 67. Python can fit a line for you. But it is good to do a few of these functions by hand to see how they work.

To begin we are going to cheat and make our lives easier and use the numpy package. This package lets us do some array operations really easily and we won't have to do for loops. I was thinking of being mean and using all for loops. But let's take advantage of python. First let's import numpy and see what we can do.

You might not have scipy.

in terminal you can do `uv add scipy`.

do NOT use pip. use uv add.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.set_printoptions(legacy='1.25') #removes the funny printing
```

```
In [2]: x=np.array([1.,3,4,6])
print (x)
type(x)
```

```
[1.  3.  4.  6.]
```

```
Out[2]: numpy.ndarray
```

So we could enter a list but instead of calling it a list we call it a numpy array. This is like a supercharged list.

Now we can make the y

```
In [3]: y=np.array([2,3,4,5])
```

Now this is where numpy gets really cool. you can multiply and add your lists. This is different than array math if you have taken linear algebra. But we will be able to do that also.

```
In [4]: print (x*y)
```

```
[ 2.  9. 16. 30.]
```

do you see what it just did? It multiplied elementwise!

```
In [5]: print (x+y)
```

```
[ 3.  6.  8. 11.]
```

```
In [6]: print (x-y)
```

```
[-1.  0.  0.  1.]
```

```
In [7]: print (x/y)
```

```
[0.5  1.   1.   1.2]
```

```
In [8]: print (x%y)
```

```
[1.  0.  0.  1.]
```

If you are doing linear algebra there are methods to do true matrix multiplication. For example the dot product.

```
In [9]: print (np.dot(x,y))
```

57.0

```
In [10]: print (np.sum(x))
```

14.0

```
In [11]: print (len(x))
```

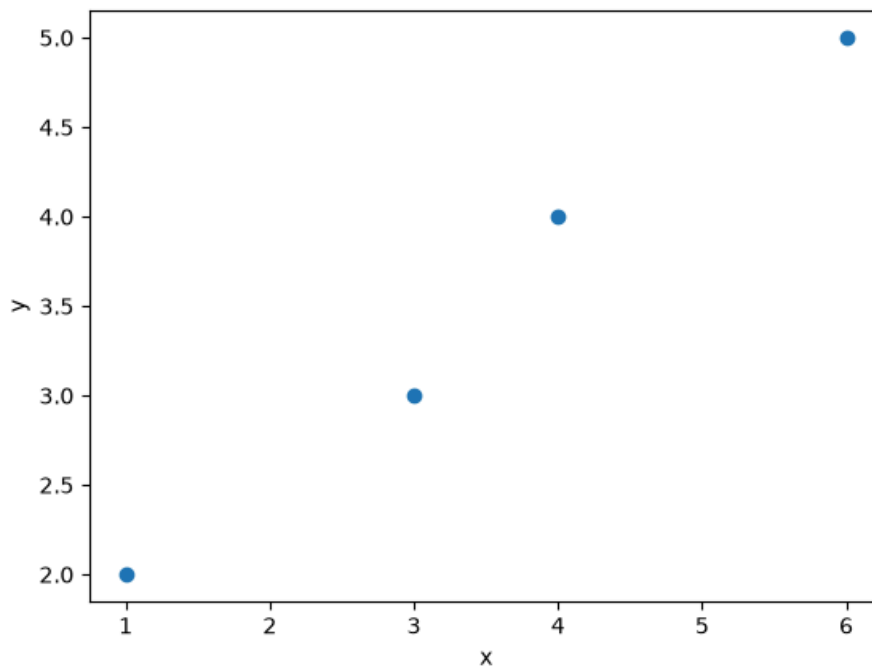
4

remember that tab is your friend. if you type np. and hit tab you will see a ton of functions we can call

```
In [13]: np.
```

What does our data look like?

```
In [14]: fig,ax=plt.subplots()  
ax.scatter(x,y)  
ax.set_xlabel('x')  
ax.set_ylabel('y')  
plt.show()
```



It is not a perfect line. So we need to fit a best fit line!

Now we can fit a line!!!!

I have now given you all the tools you need to figure out the best fit equation of a line. Remember the best fit equation of the line is $y=mx+b$ where m is the slope and b is the intercept. We fit 2 points last time which define a line. When you have many points you have to find the best fit. We can do that! To do the fit when you have multiple points you get the equations

$$m = \frac{(n\sum x_i y_i - \sum x_i \sum y_i)}{n\sum (x_i^2) - (\sum x_i)^2}$$

$$b = \frac{\sum(x_i^2)\sum y_i - (\sum x_i y_i)\sum x_i}{n\sum(x_i^2) - (\sum x_i)^2}$$

remember x_i means for each element in the list of x.

so in our case $x_0 = 1, x_1 = 3, x_2 = 4, x_3 = 6$

if we sum up all of one list that is $\sum x_i = 1 + 3 + 4 + 6 = 14$

n is the length of our list

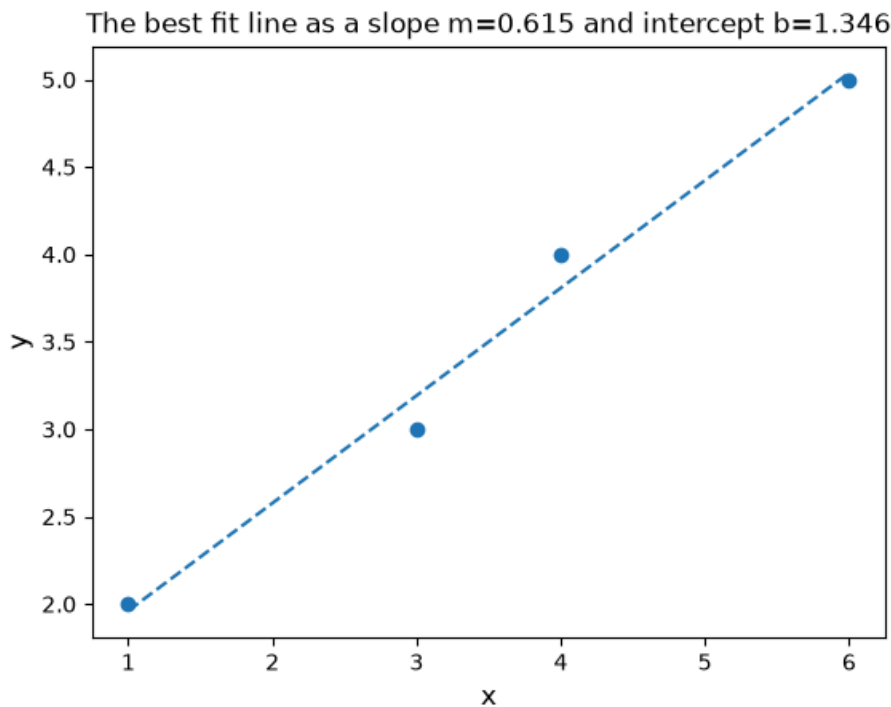
Remembr PEMDAS!

So lets plot our x and y data, look at it and then fit it.

So go ahead and figure out your m and b and then plot the line on the graph.

```
In [86]: x=np.array([1.,3,4,6])
y=np.array([2,3,4,5])

##### FILL IN MATH AND GRAPHS HERE
```



Now lets use Python to fit the line.

Two functions

1. linregress

2. Polyfit (not critical for now)

Linregress (short for linear regression)

I like linregress from scipy for my basic line fitting. the strength it has over polyfit is that it returns a p-value and an r which you can convert into r^2 along with the slope and intercept. Lets learn about it!

```
In [12]: ?stats.linregress
```

The key is we give it an x and y and then it returns:

slope : float slope of the regression line

intercept : float intercept of the regression line

r-value : float correlation coefficient

p-value : float two-sided p-value for a hypothesis test whose null hypothesis is that the slope is zero.

stderr : float Standard error of the estimate

We have talked about this some. But I want to say more explicitly here. In python when you call a function it can return many things. It doesn't have to return one number. It can return an array or multiple values. For linregress it returns 5 values. We can name these or put them in an array (I use array and list semi-interchangeably, I apologize and I will try to fix this). **HOW PYTHON CAN RETURN MANY THINGS ON THE LEFT SIDE OF AN EQUAL SIGN IS WEIRD.** Get used to it!

```
In [23]: stats.linregress(x,y)
```

```
Out[23]: LinregressResult(slope=0.6153846153846154, intercept=1.3461538461538458, rvalue=0.9922778767136677, pvalue=0.007722123286332259, stderr=0.05439282932204183, intercept_stderr=0.21414478318576893)
```

Why do I like stats.linregress? Because it gives us the r-value (square it and you have the r-squared) and the p-value. How do these results compare against yours that you calculated?

But if you want to use your stats results set it equal to something, it will make a list and then you can access it. Or you can set each item. so the two ways are.

First way. Set results equal to a list

```
In [27]: LinResult=stats.linregress(x,y)
```

```
In [28]: print(LinResult[0])  
0.6153846153846154
```

```
In [29]: LinResult
```

```
Out[29]: LinregressResult(slope=0.6153846153846154, intercept=1.3461538461538458, rvalue=0.9922778767136677, pvalue=0.007722123286332259, stderr=0.05439282932204183, intercept_stderr=0.21414478318576893)
```

I just learned a cool trick that I like. You can also use dot notation with your linregress output! This is nice!

But now you can use the names to get the results.

```
In [30]: LinResult.slope
```

```
Out[30]: 0.6153846153846154
```

```
In [31]: print (LinResult.intercept)  
1.3461538461538458
```

We can give the output meaningful names!

```
In [32]: slope, intercept, r_value,p_value,stderr= stats.linregress(x,y)
```

```
In [33]: slope
```

Out[33]: 0.6153846153846154

```
In [34]: intercept
```

Out[34]: 1.3461538461538458

We can give the output nonsensical names. Remember we control the computer and we are naming them!

```
In [35]: phineas, ferb, perry, candace, isabelle = stats.linregress(x, y)
```

```
In [36]: print (ferb)
```

1.3461538461538458

This year I am going to try and be consistent and do

LinResult

I think it will be cleanest and give us the least change for error.

Side Note

If you have a long text string. You can sometimes get errors breaking it over multiple lines. Use the backslash One thing I have problems with is long lines I want on multiple lines. For example sometimes I like to define a long string and then use that string as a title. To have it go over multiple lines you can use brackets. here is an example of a long string I made for a title. You can see me accessing the results both ways. Plus I added a `\n` to break lines and python let me break up the code on multiple lines since I was in a parantheses. Sometimes you can also add a `\` to break the lines to get a line continuation. see <https://stackoverflow.com/questions/4172448/is-it-possible-to-break-a-long-line-to-multiple-lines-in-python>

```
In [44]: title=('MY best fit line has a slope m={:.3f} and intercept b={:.3f}'\
              .format(m,b)+
              '\nbest fit line linregress slope m={:.3f} and intercept b={:.3f} '\
              .format(LinResult.slope,LinResult.intercept))
print (title)
```

MY best fit line has a slope m=0.615 and intercept b=1.346
best fit line linregress slope m=0.615 and intercept b=1.346

Now lets fix up our graph!

We can put the title back on.

```
In [46]: x=np.array([1.,3,4,6])
         y=np.array([2,3,4,5])

         fig,ax=plt.subplots()
         fig.set_size_inches(6,6) # I made a square graph

         ax.scatter(x,y) #Usual the data is the scatter

         #find the stats
         LinResult=stats.linregress(x,y)

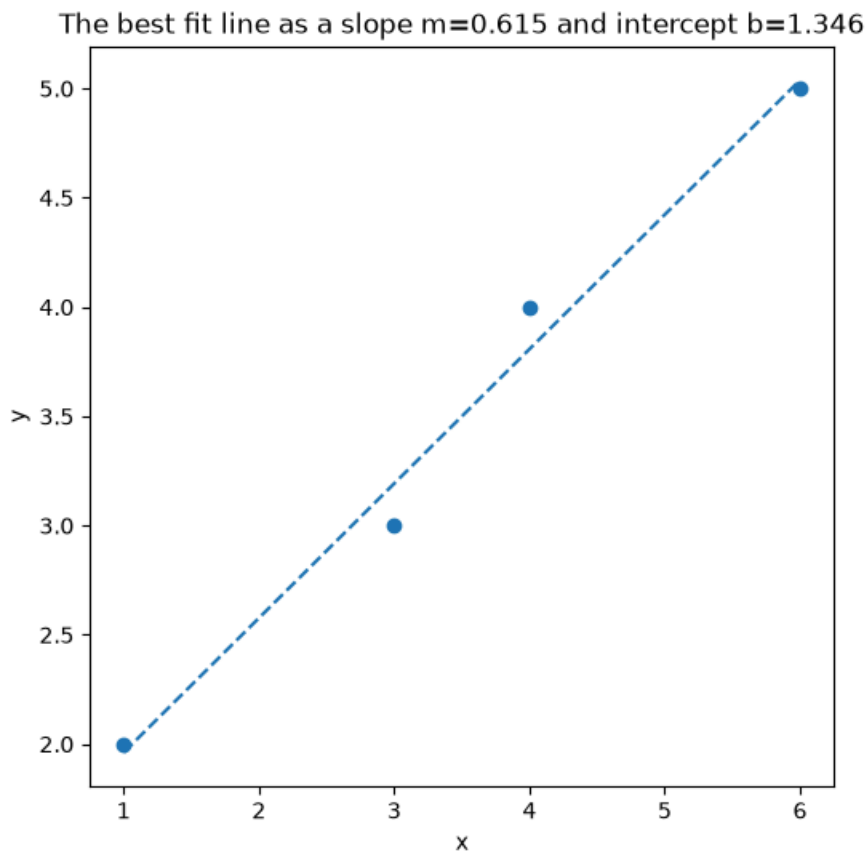
         #plot the best fit line
         x_fit=np.linspace(np.min(x),np.max(x))
         ax.plot(x_fit,x_fit*LinResult.slope+LinResult.intercept,linestyle='--')

         ax.set_xlabel('x')
         ax.set_ylabel('y')

         title=('The best fit line as a slope m={:.3f} and intercept b={:.3f}'\
```

```
.format(LinResult.slope,LinResult.intercept))
ax.set_title(title)
```

Out[46]: Text(0.5, 1.0, 'The best fit line as a slope m=0.615 and intercept b=1.346')



But I think adding a textbox to the graph makes it look more professional

Sometimes when making a graph, instead of putting in a title it looks better to put in a text box with just the details. It is a three step process to make a nice box. Scroll down this link and you can see where I got the recipe from.

<http://matplotlib.org/users/recipes.html> It is at the bottom where it says Placing Text Boxes

1. First you define the box by making a dictionary of the box properties. We usually call it props for the properties of the box.
2. Then you make the text string you want in the box. for a linear equation you usually want slope, intercept, r^2 , and p-value
3. You then say where you want the information. This is within the ax properties since we will put it into the graph. You tell it the relative location, Then you give it the text, some more information, and then the props
4. Also add the linregress to this cell to do everything in one place to make it clean

```
In [67]: x=np.array([1.,3,4,6])
y=np.array([2,3,4,5])

fig,ax=plt.subplots()
fig.set_size_inches(6,6) # I made a square graph

ax.scatter(x,y,s=125,alpha=0.8)

#Stats on the data
LinResult=stats.linregress(x,y)

#plot the best fit line
```

```

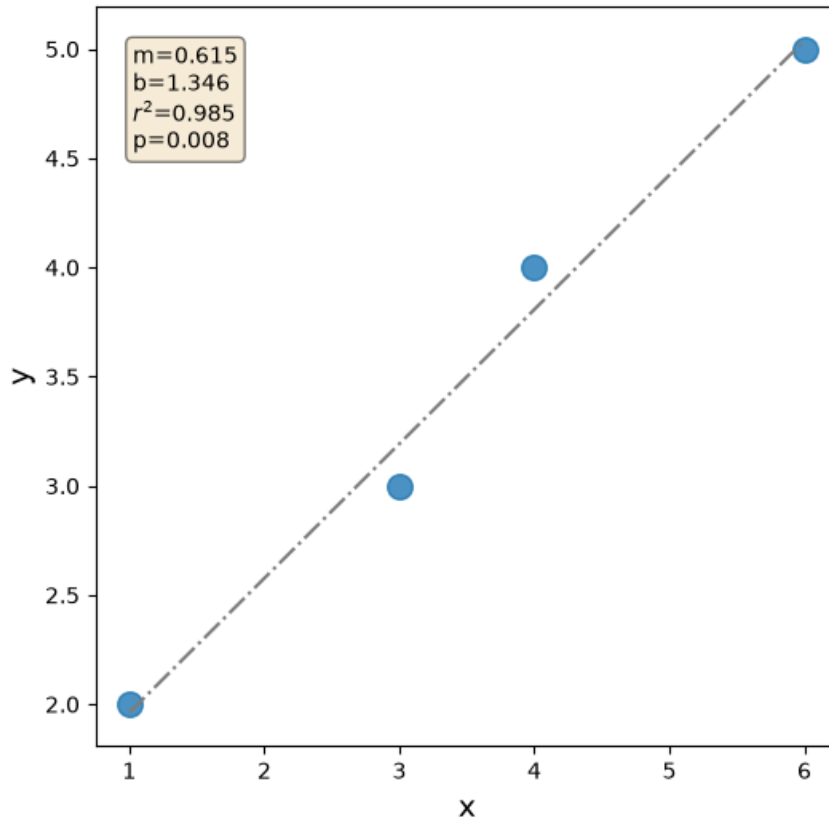
x_fit=np.linspace(np.min(x),np.max(x))
ax.plot(x_fit,x_fit*LinResult.slope+LinResult.intercept,ls='-.',color='grey')

ax.set_xlabel('x',fontsize=14)
ax.set_ylabel('y',fontsize=14)

# This is the code I added to get the box below with the normal graphing
props=dict(boxstyle='round',facecolor='wheat',alpha=0.5)

textstr='m={:.3f}\nb={:.3f}\nr^2$={:.3f}\np={:.3f}'\
        .format(LinResult.slope, LinResult.intercept
                ,LinResult.rvalue**2, LinResult.pvalue)
ax.text(0.05,0.95,textstr,transform=ax.transAxes\
        ,fontsize=10,verticalalignment='top',bbox=props)
plt.show()

```



In []:

Class Assignment

Fit a line to the KLGA and KNYC data and plot it

Now can you go back and get the KNYC and KLGA weather data and see if they are correlated? I would use your program and then compare to linregress. Remember to use `np.array([])` to enter the data as a numpy array. Also remember you need at least one float in your list to make it all floats. I like the second graph with the box! Here is the data I used for my graphs

```

In [ ]: KLGA=np.array([52,43,39,34,33,31,34])
        KNYC=np.array([53,44,40,37,36,30,32])

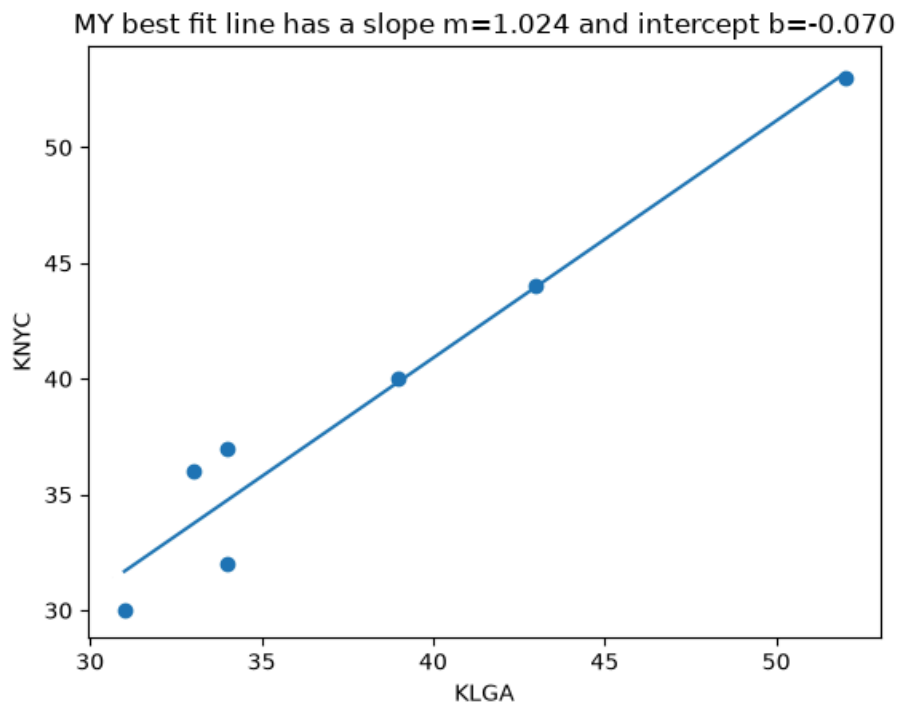
```

In [87]:

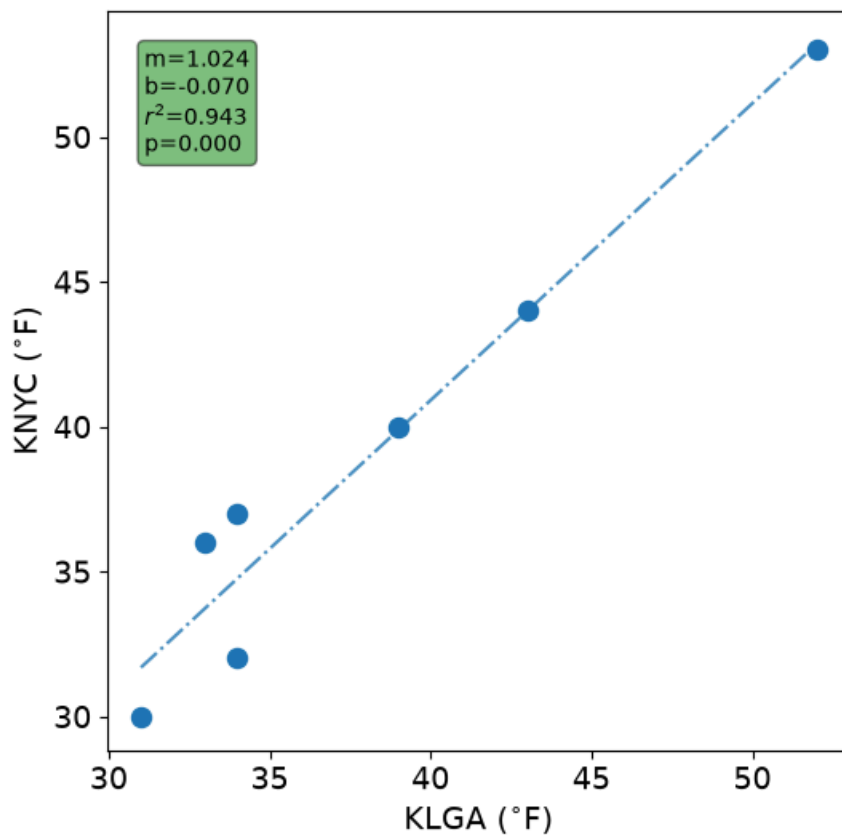
```

Out[87]: Text(0.5, 1.0, 'MY best fit line has a slope m=1.024 and intercept b=-0.070')

```



In [61]:



This is Bonus material to help you understand p-values. If you have time left after linregress keep going with this. But not on HW.

What is a p-value?

Statisticians argue about p-values and what they exactly mean. We are going to do an exercise to help you understand.

In my simplistic world I think of a p-value as the chance of the result happening by chance.

a p-value of 0.05 means that result could happen by chance 5% of the time. It sort of but not quite means that the result is real 95% of the time.

a p-value of 0.01 means that result could happen by chance 1% of the time. It sort of but not quite means that the result is real 99% of the time.

you usually see people writing $p < 0.05$ when they want to show the relationship is significant. We say it is significant because only 5% of the time it happens randomly.

This means if we randomly create data. 5% of the time we would get a $p\text{-value} < 0.05$. 95% of the time our results would look like junk. so lets do it!

Use the numpy function random to get random numbers from 0 to 1. it is `np.random.random(size)` and you give how many. lets do 50.

```
In [46]: np.random.random(50)
```

```
Out[46]: array([0.43494655, 0.18212523, 0.64058875, 0.1344949 , 0.01361897,
 0.71982023, 0.29131241, 0.70704831, 0.49865996, 0.44239256,
 0.28538041, 0.40703155, 0.80541376, 0.39869119, 0.51059245,
 0.54187569, 0.6867039 , 0.21420721, 0.87998121, 0.86427186,
 0.84706756, 0.59461833, 0.18973756, 0.44106263, 0.04924974,
 0.63475713, 0.59643627, 0.21474016, 0.8161743 , 0.7502081 ,
 0.44468346, 0.06310597, 0.32852646, 0.39601066, 0.22105364,
 0.56186312, 0.15504018, 0.61923587, 0.00742019, 0.64527563,
 0.5493542 , 0.22118838, 0.27823596, 0.55668109, 0.79909457,
 0.64948811, 0.8439744 , 0.84705429, 0.28300244, 0.51041085])
```

Now make your x and y data each with 50 numbers

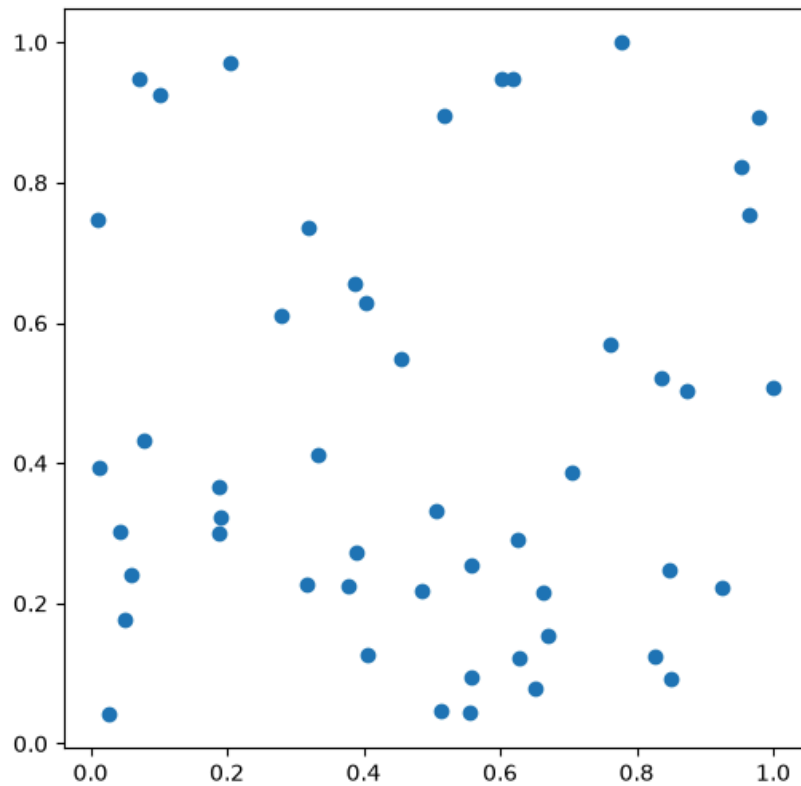
```
In [47]: x=np.random.random(50)
y=np.random.random(50)
```

Now plot the data. Every time you run it your data will change. Run it a few times and see if the points move around!

```
In [64]: x=np.random.random(50)
y=np.random.random(50)

fig,ax=plt.subplots()
fig.set_size_inches(6,6) # I made a square graph
ax.scatter(x,y)

LinResult= stats.linregress(x,y)
```



Now add the best fit line

```
In [79]: x=np.random.random(50)
y=np.random.random(50)

fig,ax=plt.subplots()
fig.set_size_inches(6,6) # I made a square graph
ax.scatter(x,y)

#Stats on the data
LinResult=stats.linregress(x,y)

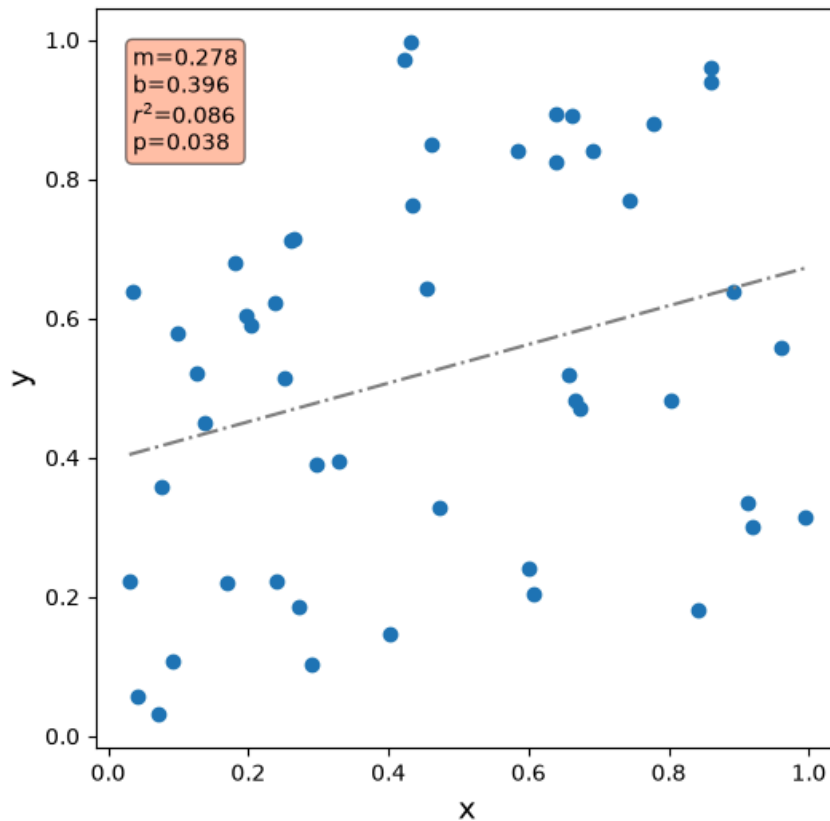
#plot the best fit line
x_fit=np.linspace(np.min(x),np.max(x))
ax.plot(x_fit,x_fit*LinResult.slope+LinResult.intercept,ls='-.',color='grey')

ax.set_xlabel('x',fontsize=14)
ax.set_ylabel('y',fontsize=14)

# This is the code I added to get the box below with the normal graphing
props=dict(boxstyle='round',facecolor='coral',alpha=0.5)

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\n$p$={:.3f}'\
        .format(LinResult.slope, LinResult.intercept
                ,LinResult.rvalue**2, LinResult.pvalue)
ax.text(0.05,0.95,textstr,transform=ax.transAxes\
        ,fontsize=10,verticalalignment='top',bbox=props)

plt.show()
```



In []:

Now rerun the cell and count how many times it takes you to get a p-value less than 0.05. Tell me what you get.

Now lets make the computer work for us. Think about this. We can ask the computer to make the graph above 1000 times. Then we can ask how many times we got a p-value less than 0.05. If it is random our answer should come about near but probably not exactly 50.

Now lets get rid of the graph and run the regression 1000 times and count how many times we get a significant result

```
In [82]: num_sig=0

for i in np.arange(1000):
    x=np.random.random(50)
    y=np.random.random(50)

    #calculate the best fit line
    LinResult= stats.linregress(x,y)
    if LinResult.pvalue<0.05:
        num_sig+=1
        print('Loop Number {} with a p_value of {}'.format(i,LinResult.pvalue))

print('I ran the for loop 1000 times and the \
p_value was less than 0.05 {} times'.format(num_sig))
```

```

Loop Number 9 with a p_value of 0.045387378058917224
Loop Number 19 with a p_value of 0.04851533679970931
Loop Number 37 with a p_value of 0.02788773666295639
Loop Number 40 with a p_value of 0.04286644761919469
Loop Number 51 with a p_value of 0.03229952593235381
Loop Number 71 with a p_value of 0.040745905796182605
Loop Number 94 with a p_value of 0.0023933929286019404
Loop Number 104 with a p_value of 0.032048865130711805
Loop Number 108 with a p_value of 0.04581033827958525
Loop Number 135 with a p_value of 0.03118267190834496
Loop Number 152 with a p_value of 0.009473592275694631
Loop Number 176 with a p_value of 0.030384336300759228
Loop Number 178 with a p_value of 0.04967045430476051
Loop Number 194 with a p_value of 0.030953227458444824
Loop Number 216 with a p_value of 0.005942055531078148
Loop Number 226 with a p_value of 0.00238691199541765
Loop Number 231 with a p_value of 0.018871921753157
Loop Number 246 with a p_value of 0.038278635749987695
Loop Number 253 with a p_value of 0.038085130443093564
Loop Number 293 with a p_value of 0.0012064514168913633
Loop Number 302 with a p_value of 0.03940276692327499
Loop Number 322 with a p_value of 0.019434998500415808
Loop Number 328 with a p_value of 0.024819514860651835
Loop Number 331 with a p_value of 0.017646099578820188
Loop Number 343 with a p_value of 0.04358025760640026
Loop Number 374 with a p_value of 0.01677398158580587
Loop Number 377 with a p_value of 0.014891588166349006
Loop Number 402 with a p_value of 0.03436579585241363
Loop Number 416 with a p_value of 0.03837714454242771
Loop Number 446 with a p_value of 0.044869219760132195
Loop Number 482 with a p_value of 0.014911460019626867
Loop Number 486 with a p_value of 0.03801775936868472
Loop Number 501 with a p_value of 0.011584172860801082
Loop Number 516 with a p_value of 0.010049129059509006
Loop Number 519 with a p_value of 0.029907163336727303
Loop Number 576 with a p_value of 0.01490561114833035
Loop Number 596 with a p_value of 0.02290523258401985
Loop Number 598 with a p_value of 0.04643513261730577
Loop Number 613 with a p_value of 0.0016597702675585767
Loop Number 636 with a p_value of 0.026369904237471355
Loop Number 643 with a p_value of 0.012224756045262209
Loop Number 645 with a p_value of 0.0014849472968775624
Loop Number 671 with a p_value of 0.043262678091762874
Loop Number 676 with a p_value of 0.033829937195046754
Loop Number 679 with a p_value of 0.01061880573289716
Loop Number 694 with a p_value of 0.04393939446139342
Loop Number 747 with a p_value of 0.002200791864453422
Loop Number 786 with a p_value of 0.016275011905918374
Loop Number 810 with a p_value of 0.023127870875352726
Loop Number 818 with a p_value of 0.018050966501869607
Loop Number 838 with a p_value of 0.04968959292773354
Loop Number 949 with a p_value of 0.00799312422062949
Loop Number 954 with a p_value of 0.02528825537459072
I ran the for loop 1000 times and the p_value was less than 0.05 53 times

```

So hopefully this helps you with a p-values. the p-value tells you how often the result may happen randomly. So the lower the p-value the lower the probability of the result happening randomly. Therefore you can "trust" the result more. But really you report the p-value so people know how you are making your choice on the significance of the results. A lot of methods report a p-value so you will be seeing this!

In []:

Fit a Line Answers

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
```

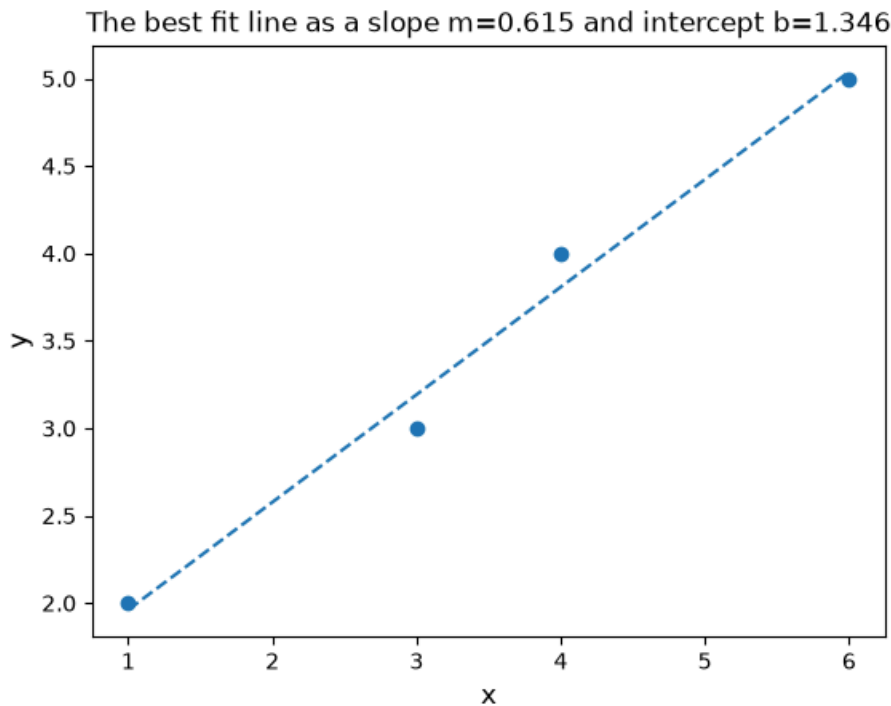
```
from scipy import stats
```

```
In [53]: x=np.array([1.,3,4,6])
y=np.array([2,3,4,5])

xy=x*y
n=len(x)
denom=n*sum(x*x)-sum(x)**2
m=(n*sum(xy)-sum(x)*sum(y))/denom
b=(sum(x*x)*sum(y)-sum(xy)*sum(x))/denom

x_fit=np.linspace(min(x),max(x))
y_fit=m*x_fit+b

fig,ax=plt.subplots()
ax.scatter(x,y)
ax.plot(x_fit,y_fit,linestyle='--')
ax.set_xlabel('x',fontsize=12)
ax.set_ylabel('y',fontsize=12)
title='The best fit line as a slope m={:.3f} and intercept b={:.3f}'.format(m,b)
ax.set_title(title)
plt.show()
```



```
In [54]: KLGA=np.array([52,43,39,34,33,31,34])
KNYC=np.array([53,44,40,37,36,30,32])

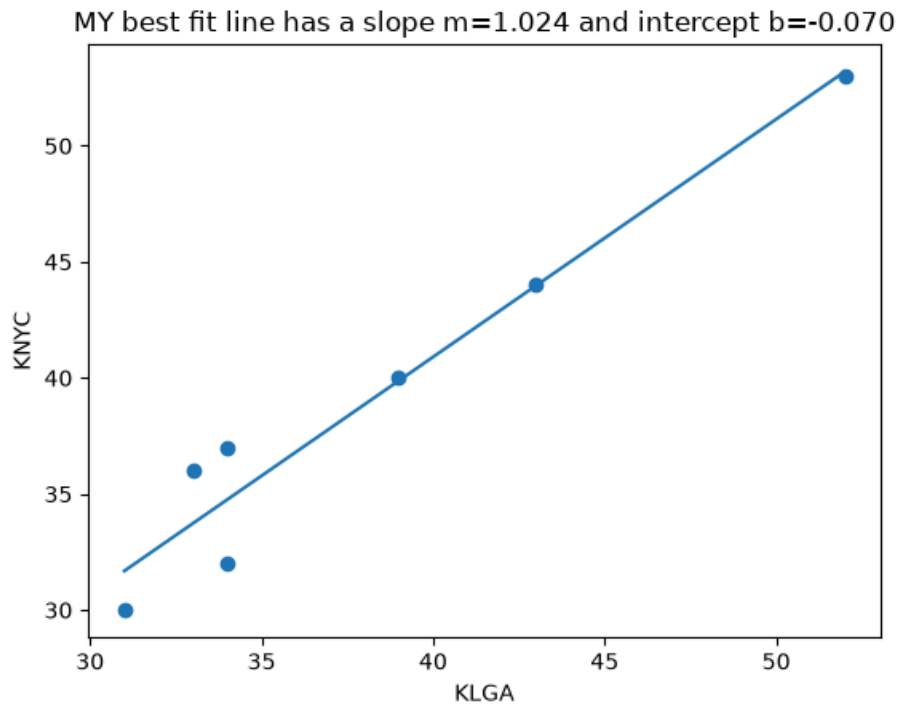
x=KLGA
y=KNYC
xy=x*y
n=len(x)
denom=n*sum(x*x)-sum(x)**2
m=(n*sum(xy)-sum(x)*sum(y))/denom
b=(sum(x*x)*sum(y)-sum(xy)*sum(x))/denom

x_fit=np.linspace(min(x),max(x))
y_fit=m*x_fit+b

fig,ax=plt.subplots()
ax.scatter(x,y)
ax.plot(x_fit,y_fit)
ax.set_xlabel('KLGA')
ax.set_ylabel('KNYC')
```

```
title=('MY best fit line has a slope m={:.3f} and intercept b={:.3f}'.format(m,b))
ax.set_title(title)
```

Out[54]: Text(0.5, 1.0, 'MY best fit line has a slope m=1.024 and intercept b=-0.070')



Using Linregress

Now that is only a decent plot. We don't have units on the axis and I will show you how to represent the m, b, r^2 and p on the graph. Here we go.

```
In [85]: KLGA=np.array([52,43,39,34,33,31,34])
KNYC=np.array([53,44,40,37,36,30,32])

fig,ax=plt.subplots()
fig.set_size_inches(6,6)
ax.scatter(KLGA,KNYC,s=75)

ax.set_xlabel('KLGA ($^{\circ}$F)',fontsize=14)
ax.set_ylabel('KNYC ($^{\circ}$F)',fontsize=14)

ax.tick_params(axis='both', which='major', labelsize=14) #This increases the font size.

LinResult=stats.linregress(KLGA,KNYC)

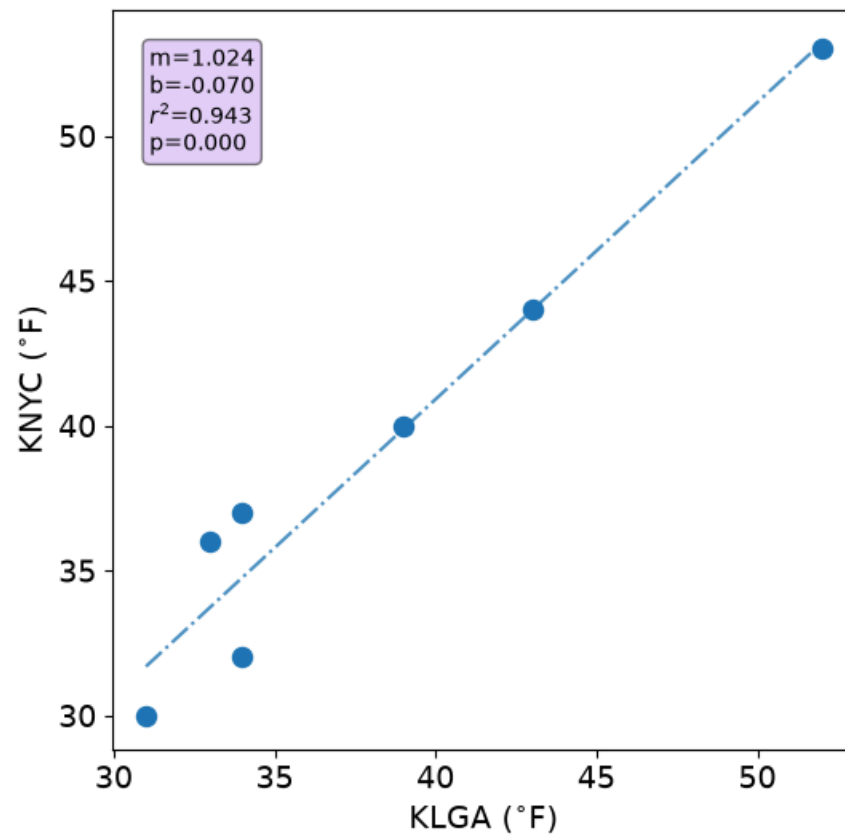
x_fit=np.linspace(min(KLGA),max(KLGA))
y_fit=LinResult.slope*x_fit+LinResult.intercept
ax.plot(x_fit,y_fit,ls='-.',alpha=0.8)

props=dict(boxstyle='round', facecolor='xkcd:lavender', alpha=0.5)

textstr='m={:.3f}\nb={:.3f}\nr^2$={:.3f}\np$={:.3f}'\
        .format(LinResult.slope,LinResult.intercept
                ,LinResult.rvalue**2,LinResult.pvalue)

ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)

plt.show()
```



Now we are starting to look professional!!!! and can make a figure caption. Figure 1. Temperature at JFK Airport (KNYC) versus temperature at Laguardia Airport (KLGA). The line is the best fit linear regression.

In []: